Tab 1

Lộ trình tối ưu hóa Kubernetes dành cho kỹ sư phát triển Java Backend: Chiến lược làm chủ hệ thống trong 24 giờ và thực tiễn vận hành doanh nghiệp

Trong bối cảnh chuyển đổi số mạnh mẽ, việc dịch chuyển từ kiến trúc monolithic sang microservices đã đặt ra những thách thức mới cho các kỹ sư phát triển Java Backend. Kubernetes (k8s) không còn là một công cụ tùy chọn mà đã trở thành nền tảng vận hành tiêu chuẩn, nơi các ứng dụng Java được triển khai, quản lý và mở rộng.[1] Đối với một lập trình viên Java, việc học Kubernetes không chỉ dừng lại ở các câu lệnh kubectl cơ bản mà đòi hỏi sự hiểu biết sâu sắc về cách thức máy ảo Java (JVM) tương tác với các cơ chế điều phối container.[3] Báo cáo này thiết lập một lộ trình học tập chuyên biệt trong 24 giờ, được thiết kế để cung cấp đầy đủ các kiến thức từ nền tảng đến thực chiến, tập trung vào việc tối ưu hóa hiệu suất, bảo mật và xây dựng quy trình CI/CD hoàn chỉnh cho hệ sinh thái Java.

Khối kiến thức 1: Nền tảng và Tiền đề kỹ thuật (Giờ 1 - Giờ 3)

Trước khi bắt đầu với Kubernetes, nhà phát triển cần làm chủ các công nghệ nền tảng tạo nên hệ sinh thái container. Kubernetes thực chất là một hệ thống điều phối, và để điều phối hiệu quả, các đơn vị cơ bản (container) phải được xây dựng đúng chuẩn.[1]

Cơ sở hạ tầng và Hệ điều hành Linux

Hầu hết các bản triển khai Kubernetes đều dựa trên Linux. Nhà phát triển Java cần hiểu về các khái niệm như hệ thống tệp (filesystems), điểm gắn kết (mount points), và đặc biệt là systemd cùng journalctl để quản lý log.[5] Các kỹ năng về mạng như hiểu biết về dải IP CIDR, các tầng OSI (L2, L3, L4, L7) và giao thức DNS là bắt buộc để xử lý các vấn đề kết nối giữa các microservices.[5]

Công nghệ Container và Docker

Container là đơn vị thực thi nhỏ nhất trong thế giới Cloud Native. Nhà phát triển phải nắm vững cách xây dựng Dockerfile, quản lý image và hiểu về Container Runtime Interface (CRI).[5] Đối với Java, việc lựa chọn image cơ sở (base image) có ảnh hưởng trực tiếp đến kích thước và tính bảo mật của ứng dụng.[6]

Bảng 1: Danh mục các kỹ năng tiền đề cho Kubernetes

Danh mục	Kỹ năng trọng tâm	Tầm quan trọng đối với Java Developer
Linux Internal	Iptables, Namespaces, Cgroups	Hiểu cách giới hạn tài nguyên CPU/RAM cho JVM [5]
Networking	DNS, SSL/TLS, Proxy	Cấu hình kết nối giữa ứng dụng Java và Database/Redis [5]
YAML/JSON	Cấu trúc dữ liệu, Syntax	Viết các tệp manifest cấu hình Pod và Deployment [5]
REST/gRPC	API Design	Giao tiếp giữa các microservices trong cụm [5]
Containerization	Docker, Multi-stage builds	Tối ưu hóa kích thước image cho ứng dụng Spring Boot [6]

Khối kiến thức 2: Kiến trúc Kubernetes và Các đối tượng cốt lõi (Giờ 4 - Giờ 7)

Kiến trúc của Kubernetes được thiết kế theo mô hình phân tán, chia thành hai phần chính: Control Plane (Mặt phẳng điều khiển) và Worker Nodes (Các nút công việc).[1]

Thành phần của Control Plane

Control Plane đóng vai trò là "bộ não" của cụm, bao gồm API Server (trung tâm giao tiếp), etcd (kho lưu trữ trạng thái), Scheduler (lập lịch cho Pod) và Controller Manager (duy trì trạng thái mong muốn).[5] Đối với nhà phát triển Java, hiểu cách API Server hoạt động giúp hình dung được cách thức các lệnh triển khai được thực hiện từ máy cục bộ hoặc từ pipeline CI/CD.[5]

Các đối tượng cơ bản trong thực tế triển khai Java

Trong Kubernetes, nhà phát triển không tương tác trực tiếp với container mà thông qua các lớp trừu tượng:

1. **Pod**: Đây là đơn vị nhỏ nhất, chứa một hoặc nhiều container chia sẻ chung không gian mạng và lưu trữ.[1] Một ứng dụng Spring Boot thông thường sẽ chạy trong một Pod đơn lẻ.
2. **Deployment**: Quản lý việc triển khai các Pod, hỗ trợ cập nhật không gián đoạn (rolling

updates) và tự động khôi phục khi Pod gặp sự cố.[1]

3. **Service**: Cung cấp một địa chỉ IP ổn định để truy cập vào nhóm các Pod. Các loại Service phổ biến bao gồm ClusterIP (nội bộ), NodePort (mở cổng trên node) và LoadBalancer (tích hợp với cloud).[3]
4. **ConfigMap và Secret**: Dùng để quản lý cấu hình và dữ liệu nhạy cảm (như mật khẩu DB) tách biệt khỏi mã nguồn.[2]

Bảng 2: So sánh các loại Service trong Kubernetes

Loại Service	Phạm vi truy cập	Trường hợp sử dụng tiêu biểu cho Java Backend
ClusterIP	Chỉ bên trong cụm	Giao tiếp giữa Microservice A và Microservice B [5]
NodePort	Truy cập qua IP của Node	Môi trường thử nghiệm hoặc tích hợp hệ thống cũ [3]
LoadBalancer	Truy cập từ Internet	Cung cấp API công khai cho ứng dụng di động/web [5]
Headless	Trực tiếp IP của Pod	Triển khai các hệ thống có trạng thái như Kafka hoặc DB [5]

Khối kiến thức 3: Tối ưu hóa Docker và Java Image (Giờ 8 - Giờ 10)

Việc container hóa ứng dụng Java một cách sơ sài có thể dẫn đến các image nặng nề (trên 1GB), thời gian build lâu và chứa nhiều lỗ hổng bảo mật.[6] Các nhà phát triển chuyên nghiệp cần áp dụng các kỹ thuật tiên tiến để đảm bảo ứng dụng "blazing fast" trong môi trường sản xuất.

Sử dụng Multi-stage Builds và Distroless Images

Chiến lược build đa giai đoạn cho phép tách biệt môi trường xây dựng (có Maven/Gradle và JDK) khỏi môi trường thực thi (chỉ cần JRE tinh gọn).[6] Việc sử dụng các "distroless images" từ Google (như gcr.io/distroless/java17-debian12) giúp loại bỏ các shell và package manager không cần thiết, giảm đáng kể bề mặt tấn công của hacker.[6]

Tối ưu hóa Layered JARs trong Spring Boot 3.x

Spring Boot 3.x hỗ trợ cấu trúc JAR phân lớp. Bằng cách bật tính năng này trong pom.xml, Docker có thể cache các lớp thư viện ít thay đổi (dependencies) và chỉ rebuild lớp mã nguồn (classes).[6] Điều này có thể giúp giảm thời gian đẩy image lên registry lên tới 10 lần trong quy trình CI/CD.[6]

Bảng 3: Hiệu quả của các phương pháp tối ưu hóa Image Java

Phương pháp	Kích thước Image	Bảo mật	Tốc độ CI/CD
Standard OpenJDK	> 600 MB	Thấp (chứa nhiều tool thừa)	Chậm [6]
Alpine JRE	~ 120 MB	Trung bình	Nhanh [6]
Distroless Java	< 100 MB	Cao nhất	Rất nhanh [6]
Layered JARs	Tương đương	Cao	Cực nhanh (nhờ cache) [6]

Khối kiến thức 4: Quản lý tài nguyên và Cơ chế JVM Aware (Giờ 11 - Giờ 13)

Một trong những vấn đề gây nhức nhối nhất khi chạy Java trên Kubernetes là tình trạng Pod bị giết do lỗi OOMKilled mặc dù heap usage vẫn ở mức thấp.[13] Điều này xuất phát từ việc JVM tiêu thụ nhiều loại bộ nhớ khác ngoài heap và sự không tương thích với các giới hạn của container (cgroups).[4]

Cơ chế Container Awareness

Kể từ Java 10+, JVM đã có khả năng nhận biết giới hạn tài nguyên của container.[13] Nhà phát triển nên sử dụng tham số -XX:+UseContainerSupport và thiết lập heap dựa trên phần trăm bộ nhớ của container thay vì con số tuyệt đối, ví dụ: -XX:MaxRAMPercentage=75.0.[4] Điều này giúp JVM tự điều chỉnh khi chúng ta thay đổi cấu hình RAM trong manifest Kubernetes mà không cần phải can thiệp vào tham số khởi động của Java.[4]

Tính toán Resource Requests và Limits

Việc thiết lập tài nguyên trong Kubernetes cần bao phủ toàn bộ dấu chân bộ nhớ (memory footprint) của JVM, bao gồm:

- **Heap Memory**: Vùng nhớ chính cho đối tượng.

- **Metaspace**: Lưu trữ metadata của class (nên giới hạn bằng -XX:MaxMetaspaceSize).
- **Code Cache**: Vùng nhớ cho mã đã JIT biên dịch.
- **Thread Stack**: Mỗi thread tiêu thụ mặc định 1MB (có thể giảm bằng -Xss512k).
- **Direct Buffers**: Vùng nhớ cho các thao tác I/O hiệu suất cao.[13]

Công thức tính toán tổng bộ nhớ yêu cầu (Memory Request) an toàn được khuyến nghị:

$$MemoryRequest = (Heap + Metaspace + Threads + Direct) \times 1.25$$

.[13] Hệ số 1.25 cung cấp 25% dung lượng đệm cho bộ nhớ native và overhead của tiến trình container.[13]

Bảng 4: Chiến lược cấu hình tài nguyên cho Pod Java

Tham số Kubernetes	Ý nghĩa	Khuyến nghị cho Java
Memory Request	Tài nguyên đảm bảo	Đặt bằng Memory Limit để tối ưu QoS (Guaranteed)
Memory Limit	Ngưỡng cắt tài nguyên	Tính toán dựa trên công thức Heap + 25% overhead
CPU Request	Trọng số lập lịch	Tương ứng với số nhân cần thiết cho GC xử lý
CPU Limit	Ngưỡng giới hạn	Có thể không đặt để tránh hiện tượng CPU Throttling

Khối kiến thức 5: Vòng đời ứng dụng và Health Checks (Giờ 14 - Giờ 16)

Kubernetes cần biết khi nào một ứng dụng Java thực sự sẵn sàng để nhận lưu lượng và khi nào nó gặp sự cố không thể tự phục hồi để thực hiện khởi động lại.[16]

Tích hợp Spring Boot Actuator với Probes

Spring Boot Actuator cung cấp các điểm cuối /actuator/health/liveness và /actuator/health/readiness được thiết kế riêng cho Kubernetes.[16]

1. **Startup Probe**: Đặc biệt quan trọng cho Java. Do JVM cần thời gian khởi tạo context và nạp class (có thể mất 30-60 giây), Startup probe giúp ngăn chặn việc liveness probe khởi

động lại container quá sớm khi nó vẫn đang trong quá trình khởi động bình thường.[17]

2. **Liveness Probe**: Chỉ dùng để phát hiện các trạng thái "chết" như deadlock. Tránh cấu hình liveness probe kiểm tra các phụ thuộc bên ngoài như Database, vì nếu DB lỗi tạm thời, toàn bộ Pod Java sẽ bị restart liên tục, làm trầm trọng thêm sự cố.[16]
3. **Readiness Probe**: Kiểm tra xem Pod đã sẵn sàng phục vụ yêu cầu chưa (đã kết nối được DB, đã nạp xong cache). Nếu thất bại, Kubernetes sẽ tạm thời loại bỏ Pod khỏi danh sách Service.[17]

Cơ chế Shutdown duyên dáng (Graceful Shutdown)

Khi một Pod bị thu hồi, Kubernetes gửi tín hiệu SIGTERM. Nếu ứng dụng Java dừng ngay lập tức, các yêu cầu đang xử lý sẽ bị ngắt quãng gây lỗi cho người dùng.[2] Spring Boot hỗ trợ cấu hình server.shutdown=graceful để ứng dụng hoàn tất các request đang thực thi trước khi đóng hoàn toàn tiến trình.[2]

Khối kiến thức 6: Lưu trữ và Mạng nâng cao (Giờ 17 - Giờ 19)

Đối với các ứng dụng Java cần lưu trữ tệp (như ảnh người dùng tải lên) hoặc cần hiệu suất mạng tối ưu, việc hiểu về Volume và Ingress là không thể thiếu.[1]

Persistent Volumes (PV) và Claims (PVC)

Kubernetes tách biệt việc quản lý lưu trữ vật lý (PV) và nhu cầu sử dụng của ứng dụng (PVC).[9] Khi Pod Java bị restart, dữ liệu trong container sẽ mất, do đó các dữ liệu quan trọng phải được ghi vào PVC được gắn kết vào một đường dẫn cụ thể.[5]

Ingress Controllers và SSL Termination

Ingress đóng vai trò là API Gateway cho cụm, quản lý việc định tuyến lưu lượng HTTP/HTTPS dựa trên hostname hoặc path.[5] Việc xử lý chứng chỉ SSL tại tầng Ingress (SSL Termination) giúp giảm tải xử lý mã hóa cho ứng dụng Java, giúp code Backend tập trung hoàn toàn vào logic nghiệp vụ.[5]

Khối kiến thức 7: Quy trình CI/CD và Tự động hóa (Giờ 20 - Giờ 22)

Một kỹ sư Java hiện đại không thể coi mình đã làm chủ Kubernetes nếu không biết cách tự động hóa việc đưa code lên cụm.[10]

Chiến lược GitHub Actions cho Java

GitHub Actions hiện là lựa chọn hàng đầu nhờ tính tích hợp cao và cấu hình YAML đơn giản.[10]

Một pipeline chuẩn bao gồm:

1. **Build**: Sử dụng actions/setup-java để biên dịch với Maven/Gradle và chạy Unit Test.[10]
2. **Scan**: Kiểm tra lỗ hổng bảo mật của thư viện (với Snyk hoặc OWASP) và chất lượng code (với SonarQube).[23]
3. **Package**: Xây dựng Docker image và đẩy lên AWS ECR hoặc Docker Hub.[10]
4. **Deploy**: Cập nhật tệp manifest hoặc sử dụng Helm để triển khai vào cụm.[20]

Quản lý triển khai với Helm Charts

Helm được ví như "Maven cho Kubernetes".[8] Thay vì quản lý hàng chục tệp YAML riêng lẻ, Helm đóng gói chúng thành một "Chart" duy nhất với các biến số linh hoạt (values.yaml). Điều này cho phép nhà phát triển sử dụng cùng một bộ manifest cho cả môi trường Dev, Staging và Production chỉ bằng cách thay đổi tệp giá trị.[1]

Bảng 5: So sánh các công cụ CI/CD phổ biến cho Kubernetes

Công cụ	Ưu điểm	Nhược điểm	Phù hợp nhất với
Jenkins	Hệ sinh thái plugin cực lớn, tùy biến vô hạn	Khó quản lý, tốn tài nguyên hạ tầng	Doanh nghiệp lớn, hệ thống legacy [21]
GitHub Actions	Tích hợp sẵn, không cần server riêng, dễ cấu hình	Phụ thuộc vào hệ sinh thái GitHub	Dự án mới, microservices hiện đại [10]
ArgoCD	Hỗ trợ GitOps tuyệt vời, tự động đồng bộ trạng thái	Chỉ tập trung vào phần CD (Deployment)	Các đội ngũ yêu cầu tính tuân thủ cao [24]

Khối kiến thức 8: Case Study - Triển khai Hệ thống Thanh toán Microservices (Thực tế)

Để áp dụng các kiến thức trên, chúng ta cùng phân tích quy trình triển khai thực tế của một ứng dụng thanh toán (Payment Service).

Mô tả bài toán: Ứng dụng xử lý giao dịch tài chính, yêu cầu tính bảo mật tuyệt đối, khả năng mở rộng nhanh khi có khuyến mãi và độ trễ thấp.

Các bước triển khai chuyên sâu:

1. **Cấu hình JVM**: Sử dụng Java 21 với Virtual Threads để xử lý hàng ngàn giao dịch đồng thời mà không tốn nhiều RAM cho thread stack.[28] Cài đặt -XX:+UseG1GC cho các Pod có RAM trên 2GB để tối ưu thời gian dừng Garbage Collection.[4]
2. **Đóng gói**: Sử dụng Cloud Native Buildpacks thông qua lệnh ./mvnw spring-boot:build-image để tạo image chuẩn hóa mà không cần viết Dockerfile thủ công.[2]
3. **Hạ tầng mạng**: Triển khai Ingress với cơ chế Rate Limiting để ngăn chặn tấn công DDoS vào API thanh toán.[5]
4. **Quản lý cấu hình**: Kết nối Database thông qua cơ chế nạp Secret vào biến môi trường SPRING_DATASOURCE_PASSWORD. Sử dụng NetworkPolicy để đảm bảo chỉ có Service xử lý đơn hàng mới được quyền gọi Service thanh toán.[2]
5. **Giám sát và Cảnh báo**: Tích hợp Prometheus để theo dõi các chỉ số JVM như Heap sử dụng, số lượng thread đang chạy. Thiết lập Alertmanager để gửi cảnh báo vào Slack nếu tỷ lệ lỗi HTTP 5xx tăng quá 1% trong 5 phút.[29]

Chẩn đoán lỗi và Gỡ lỗi nâng cao (Troubleshooting Giờ 23 - Giờ 24)

Kỹ năng cuối cùng để trở thành một chuyên gia là khả năng "bắt bệnh" hệ thống khi có sự cố.

Luồng chẩn đoán chuẩn: Get -> Describe -> Logs

Khi một Pod Java không hoạt động, nhà phát triển thực hiện:

- kubectl get pods: Kiểm tra xem Pod có bị CrashLoopBackOff không.[8]
- kubectl describe pod <name>: Xem phần Events. Nếu thấy OOMKilled, chắc chắn limit bộ nhớ quá thấp. Nếu thấy FailedScheduling, cụm có thể đã hết tài nguyên.[8]
- kubectl logs <name>: Truy cập log của JVM để tìm các ngoại lệ (Exceptions) trong code.[8]

Remote Debugging và Profiling

Nếu lỗi chỉ xảy ra trên môi trường Production, nhà phát triển có thể sử dụng JDWP (Java Debug Wire Protocol) kết hợp với kubectl port-forward để gắn debugger từ IntelliJ trực tiếp vào Pod đang chạy trong cụm.[33] Đối với các vấn đề về hiệu năng, việc thực hiện kubectl exec để chạy lệnh jmap -dump:live,format=b,file=/tmp/heap.hprof 1 cho phép trích xuất heap dump để phân tích rò rỉ bộ nhớ bằng công cụ Eclipse MAT.[14]

Bảng 6: Các mã lỗi phổ biến và cách xử lý trong Kubernetes

Mã lỗi / Trạng thái	Nguyên nhân tiềm ẩn	Giải pháp đề xuất
CrashLoopBackOff	Lỗi runtime Java, thiếu biến	Kiểm tra logs ứng dụng

	môi trường	
OOMKilled (137)	JVM vượt giới hạn RAM của container	Tăng RAM Limit hoặc giảm Heap Size (Xmx)
ImagePullBackOff	Sai tên image hoặc thiếu quyền truy cập registry	Kiểm tra imagePullSecrets
Pending	Cụm hết tài nguyên CPU/RAM hoặc lỗi node	Thêm node hoặc giảm Resource Requests
ErrImagePull	Lỗi mạng khi kéo image hoặc sai tag	Kiểm tra kết nối Internet và Docker tag

Kết luận và Định hướng tương lai

Lộ trình 24 giờ này cung cấp một khung xương vững chắc cho các nhà phát triển Java trong việc làm chủ Kubernetes. Tuy nhiên, hệ sinh thái Cloud Native luôn không ngừng biến đổi. Trong giai đoạn 2025-2026, các xu hướng như Serverless Java (với GraalVM Native Image), GitOps tự động hóa hoàn toàn và việc áp dụng AI vào quản lý chi phí cụm (như StormForge) sẽ trở thành những chủ đề quan trọng tiếp theo.[4]

Để thành công, nhà phát triển không chỉ cần nắm vững các câu lệnh mà phải xây dựng được một tư duy "Cloud Native" - nơi ứng dụng được thiết kế để có thể sụp đổ và tự phục hồi bất cứ lúc nào, cấu hình được tách biệt hoàn toàn khỏi mã nguồn, và mọi quy trình triển khai đều được tự động hóa tối đa.[3] Việc tuân thủ các thực hành tốt nhất về bảo mật, tối ưu hóa tài nguyên JVM và thiết lập hệ thống quan sát (observability) toàn diện sẽ là chìa khóa giúp các kỹ sư Java Backend xây dựng những hệ thống phân tán mạnh mẽ, tin cậy và hiệu quả trong kỷ nguyên điện toán đám mây.[29]

Nguồn trích dẫn

1. Kubernetes Roadmap: A Complete Guide [2025 Updated] - GeeksforGeeks, truy cập vào tháng 3 28, 2026, https://www.geeksforgeeks.org/blogs/kubernetes-roadmap/
2. Getting Started | Spring on Kubernetes, truy cập vào tháng 3 28, 2026, https://spring.io/guides/topicals/spring-on-kubernetes/
3. I Tried 15+ Kubernetes Courses on Udemy: Here Are My Top 6 Recommendations - Medium, truy cập vào tháng 3 28, 2026, https://medium.com/javarevisited/i-tried-15-kubernetes-courses-on-udemy-here-are-my-top-6-recommendations-649b89adb81b
4. A Practical Guide To Kubernetes And JVM Integration - Xebia, truy cập vào tháng

3 28, 2026, https://xebia.com/blog/guide-kubernetes-jvm-integration/
5. How to Learn Kubernetes in 2026 (Roadmap & Resources) - DevOps Cube, truy cập vào tháng 3 28, 2026, https://devopscube.com/learn-kubernetes-complete-roadmap/
6. Dockerize Spring Boot Like a Pro: 2025 Best Practices for Blazing ..., truy cập vào tháng 3 28, 2026, https://javascript.plainenglish.io/dockerize-spring-boot-like-a-pro-2025-best-practices-for-blazing-fast-deployments-1cd4d00fa229
7. Resource Management for Pods and Containers - Kubernetes, truy cập vào tháng 3 28, 2026, https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
8. The Ultimate Kubernetes Cheat Sheet: Essential kubectl Commands ..., truy cập vào tháng 3 28, 2026, https://www.splunk.com/en_us/blog/learn/kubernetes-commands-cheat-sheet.html
9. Kubernetes : Your Ultimate Cheatsheet - DEV Community, truy cập vào tháng 3 28, 2026, https://dev.to/kaustubhyerkade/kubernetes-your-ultimate-cheatsheet-kl6
10. GitHub Actions CI/CD Pipeline for Kubernetes - Devtron, truy cập vào tháng 3 28, 2026, https://devtron.ai/blog/create-ci-cd-pipelines-with-github-actions-for-kubernetes-the-definitive-guide/
11. Day 7: Your Kubernetes Learning Roadmap — What's Next After the Basics? - KodeKloud, truy cập vào tháng 3 28, 2026, https://kodekloud.com/blog/day-7-your-kubernetes-learning-roadmap-whats-next-after-the-basics/
12. Kubectl Cheat Sheet: A Quick Reference for Common Commands - Trilio, truy cập vào tháng 3 28, 2026, https://trilio.io/kubernetes-best-practices/kubectl-cheat-sheet/
13. How to Calculate Container Memory Overhead for Java Applications ..., truy cập vào tháng 3 28, 2026, https://oneuptime.com/blog/post/2026-02-09-container-memory-overhead-java-kubernetes/view
14. How to Diagnose Kubernetes Container Memory Leaks Using Memory Profiling Tools, truy cập vào tháng 3 28, 2026, https://oneuptime.com/blog/post/2026-02-09-container-memory-leaks-profiling/view
15. Best Practices for configuring the JVM in a Kubernetes environment | AWS Builder Center, truy cập vào tháng 3 28, 2026, https://builder.aws.com/content/2y4rrgs9rUiiyBZsR2TVLmGhNS3/best-practices-for-configuring-the-jvm-in-a-kubernetes-environment
16. Kubernetes Readiness and Liveness Probes in Spring Boot Apps - Medium, truy cập vào tháng 3 28, 2026, https://medium.com/@AlexanderObregon/kubernetes-readiness-and-liveness-probes-in-spring-boot-apps-71b6e639f869
17. Liveness, Readiness, and Startup Probes - Kubernetes, truy cập vào tháng 3 28, 2026,

https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/
18. How to Configure Spring Boot Actuator Health Checks for Cloud Run Startup - OneUptime, truy cập vào tháng 3 28, 2026, https://oneuptime.com/blog/post/2026-02-17-how-to-configure-spring-boot-actuator-health-checks-for-cloud-run-startup-and-liveness-probes/view
19. How to Build Health Probes for Kubernetes in Spring Boot - OneUptime, truy cập vào tháng 3 28, 2026, https://oneuptime.com/blog/post/2026-01-25-health-probes-kubernetes-spring-boot/view
20. CI/CD for Kubernetes using GitHub Actions & Helm | by Akash Hardia | Jan, 2026 | Medium, truy cập vào tháng 3 28, 2026, https://medium.com/@akashhardia/ci-cd-for-kubernetes-using-github-actions-helm-492d2cfb11ff
21. From Jenkins to GitHub Actions: Automating CI/CD Pipelines for Java Microservices | by Maanvik Gupta T | Medium, truy cập vào tháng 3 28, 2026, https://medium.com/@maanvik.gupta25/from-jenkins-to-github-actions-automating-ci-cd-pipelines-for-java-microservices-8c73ca291756
22. OswaldAKs/CICD-Pipeline-with-GithubActions-Docker: Deploy springboot-rest-api by github actions and docker, truy cập vào tháng 3 28, 2026, https://github.com/OswaldAKs/CICD-Pipeline-with-GithubActions-Docker
23. A Sample Spring Boot CI/CD pipeline with Jenkins. - GitHub, truy cập vào tháng 3 28, 2026, https://github.com/AbderrahmaneOd/Spring-Boot-Jenkins-CI-CD
24. Pshar10/spring-boot-app-cicd-pipeline - GitHub, truy cập vào tháng 3 28, 2026, https://github.com/Pshar10/spring-boot-app-cicd-pipeline
25. Kubernetes CI/CD with Github Actions and Helm - Gbenga Oni, truy cập vào tháng 3 28, 2026, https://gbengaoni.com/blog/Kubernetes-CI-CD-with-Github-Actions-and-Helm
26. Deploy apps on Kubernetes with GitHub Actions - from start to finish - koslib, truy cập vào tháng 3 28, 2026, https://www.koslib.com/posts/deploy-k8s-apps-helm-complete/
27. Setup Automated CI/CD pipeline using GitHub Actions and ArgoCD, truy cập vào tháng 3 28, 2026, https://kubernetes-tutorial.schoolofdevops.com/argo_adv_setup/
28. Spring Boot in 2024: A Comprehensive Review | by Master Spring Ter - Medium, truy cập vào tháng 3 28, 2026, https://master-spring-ter.medium.com/spring-boot-in-2024-a-comprehensive-review-bc9eb823e991
29. Kubernetes Best Practices in 2025: Scaling, Security, and Cost Optimization - KodeKloud, truy cập vào tháng 3 28, 2026, https://kodekloud.com/blog/kubernetes-best-practices-2025/
30. Automate java performance troubleshooting with AI-Powered thread dump analysis on Amazon ECS and EKS | Containers, truy cập vào tháng 3 28, 2026, https://aws.amazon.com/blogs/containers/automate-java-performance-troubleshooting-with-ai-powered-thread-dump-analysis-on-amazon-ecs-and-eks/
31. Troubleshooting a Java Application on Kubernetes: A Step-by-Step Guide for

SREs, truy cập vào tháng 3 28, 2026, https://ayushaarav.medium.com/troubleshooting-a-java-application-on-kubernetes-a-step-by-step-guide-for-sres-93565f6fddef
32. Debug Pods - Kubernetes, truy cập vào tháng 3 28, 2026, https://kubernetes.io/docs/tasks/debug/debug-application/debug-pods/
33. How to Set Up Remote Debugging for Java Applications Running - OneUptime, truy cập vào tháng 3 28, 2026, https://oneuptime.com/blog/post/2026-02-09-remote-debugging-java-kubernetes/view
34. How to Remote Debug A Java Application In a Kubernetes Pod | by Amrut Prabhu | Medium, truy cập vào tháng 3 28, 2026, https://amrutprabhu.medium.com/how-to-remote-debug-a-java-application-in-a-kubernetes-pod-ff0305f55f93
35. Remote debugging of Java apps in Kubernetes - DEV Community, truy cập vào tháng 3 28, 2026, https://dev.to/devlix-blog/remote-debugging-of-java-apps-in-kubernetes-2mo
36. K8s Pod Java Dump: A Hands-On Guide with Examples - CloudBolt, truy cập vào tháng 3 28, 2026, https://www.cloudbolt.io/kubernetes-cost-optimization/k8s-pod-java-dump/